PySpark DataFrame & pyspark.sql

A commented, example-driven guide — from SparkSession to window functions

This guide walks through the core PySpark DataFrame API and the **pyspark.sql** module using a small employee sales dataset (`employee_sales.csv`). Every example is shown as commented code next to a line-by-line explanation, so you can see exactly what each statement does.

Sample dataset used throughout this guide:

emp_id	name	department	region	sales_amount	sale_date
101	Ahmed Hassan	Electronics	Cairo	4500.75	2024-01-05
102	Sara Mostafa	Clothing	Alexandria	1200.00	2024-01-06
103	Omar Khaled	Electronics	Giza	3300.50	2024-01-07
...	...	...	...	...	...

1. Getting Started — SparkSession

Every PySpark program starts by creating a **SparkSession**: the entry point for creating DataFrames, reading files, and running SQL queries.

```
from pyspark.sql import SparkSession # import the SparkSession class

spark = SparkSession.builder \
.appName("EmployeeSalesApp") \
.master("local[*]") \
.getOrCreate()

print(spark.version) # confirm Spark is running
```

1. Imports **SparkSession**, the main class used to interact with Spark.
2. **.builder** starts a configuration chain for creating the session.
3. **.appName()** sets a name for the app, shown in the Spark UI.
4. **.master("local[*]")** runs Spark locally using all CPU cores.
5. **.getOrCreate()** reuses an existing session or creates a new one.
6. Prints the installed Spark version to verify the setup works.

Reading the CSV file into a DataFrame

```
df = spark.read \
.option("header", True) \
.option("inferSchema", True) \
.csv("employee_sales.csv")

df.show(5) # display first 5 rows
df.printSchema() # display column names and types
```

1. **spark.read** starts building a DataFrameReader for loading data.
2. **option("header", True)** tells Spark the first row holds column names.
3. **option("inferSchema", True)** lets Spark guess column data types.
4. **.csv(path)** reads the file and returns a DataFrame.
5. **.show(5)** prints the first 5 rows in a formatted table.
6. **.printSchema()** prints the column names, types, and nullability.

Resulting schema (sample output):

column	inferred type
emp_id	integer
name	string
department	string
region	string
sales_amount	double
sale_date	date

2. Core DataFrame Operations

2.1 Selecting and filtering columns

```
from pyspark.sql.functions import col # column reference helper

electronics = df.select("name", "department", "sales_amount") \
.filter(col("department") == "Electronics") \
.orderBy(col("sales_amount").desc())

electronics.show()
```

1. Imports **col()**, used to refer to columns in expressions.
2. **.select()** keeps only the listed columns.
3. **.filter()** keeps rows where department equals "Electronics".
4. **.orderBy(...desc())** sorts rows by sales_amount, highest first.
5. **.show()** displays the resulting filtered/sorted DataFrame.

2.2 Adding and transforming columns

```
from pyspark.sql.functions import round as spark_round

df2 = df.withColumn(
"sales_with_tax", # new column name
spark_round(col("sales_amount") * 1.14, 2) # add 14% VAT
)

df2.select("name", "sales_amount", "sales_with_tax").show(5)
```

1. Imports **round()** (aliased) to round numeric results.
2. **.withColumn(name, expr)** adds a new computed column.
3. First argument: the name of the new column.
4. Second argument: multiplies sales_amount by 1.14 and rounds to 2dp.
5. Selects and shows only the relevant columns for the first 5 rows.

2.3 Grouping and aggregating

```
from pyspark.sql.functions import sum as spark_sum, avg, count

summary = df.groupBy("department") \
.agg(
spark_sum("sales_amount").alias("total_sales"),
avg("sales_amount").alias("avg_sales"),
count("*").alias("num_sales")
) \
.orderBy(col("total_sales").desc())

summary.show()
```

1. Imports aggregate functions: sum, average, and count.
2. **.groupBy("department")** groups rows sharing the same department.
3. **.agg()** applies one or more aggregate functions per group.
4. Sums sales_amount per group, renamed to *total_sales*.
5. Averages sales_amount per group, renamed to *avg_sales*.
6. Counts rows per group, renamed to *num_sales*.
7. Closes the **.agg()** call.
8. Sorts departments by total sales, descending.
9. Displays the aggregated summary table.

department	total_sales	avg_sales	num_sales
Electronics	26650.45	5330.09	5
Clothing	6046.05	1209.21	5
Grocery	2720.70	544.14	5

3. pyspark.sql — Running SQL Queries

The **pyspark.sql** module lets you register a DataFrame as a temporary view and query it using standard SQL syntax through **spark.sql()**.

```
df.createOrReplaceTempView("sales") #
register df as SQL table "sales"

result = spark.sql("""
SELECT department,
ROUND(SUM(sales_amount), 2) AS total_sales
FROM sales
GROUP BY department
ORDER BY total_sales DESC
""")

result.show()
```

1. Registers the DataFrame as a temporary SQL view named "sales".
2. Starts building the SQL query string (triple-quoted for multiple lines).
3. **spark.sql()** parses and runs the SQL text, returning a DataFrame.
4. Selects department and the rounded, summed sales_amount.
5. Names the summed column *total_sales*.
6. Reads from the "sales" view created above.
7. Groups rows by department, just like the DataFrame API version.
8. Sorts results by total_sales in descending order.
9. Closes the SQL string.
10. Displays the query result as a DataFrame.

3.1 Row objects and pyspark.sql.types

```
from pyspark.sql import Row # build rows
manually
from pyspark.sql.types import (
StructType, StructField, StringType,
DoubleType
)

schema = StructType([
StructField("name", StringType(), True), #
nullable string
StructField("bonus", DoubleType(), True), #
nullable double
])

bonus_data = [Row(name="Ahmed Hassan",
bonus=250.0)]
bonus_df = spark.createDataFrame(bonus_data,
schema)
bonus_df.show()
```

1. **Row** creates lightweight, named-field records for a DataFrame.
2. Imports schema-building classes from **pyspark.sql.types**.
3. **StructType** wraps a list of field definitions (the schema).
4. Closing the import parenthesis.
5. Defines the schema as a list of **StructField** objects.
6. Field "name": string type, allows null values.
7. Field "bonus": double (decimal) type, allows null values.
8. Closes the StructType list.
9. Creates one Row instance representing a bonus record.
10. **spark.createDataFrame()** builds a DataFrame from rows + schema.
11. Displays the new bonus DataFrame.

4. Joining DataFrames

```
joined = df.join(
bonus_df, # right-hand DataFrame
on="name", # join key present in both
DataFrames
how="left" # keep all rows from df (left
side)
)

joined.select("name", "department",
"sales_amount", "bonus").show()
```

1. **.join()** combines df with bonus_df into a single DataFrame.
2. First argument: the DataFrame being joined to.
3. **on="name"** matches rows where the "name" column is equal.
4. **how="left"** keeps every row from df, filling unmatched bonus with null.
5. Closes the join call.
6. Selects and displays only the relevant joined columns.

5. Window Functions

Window functions compute values across a set of rows related to the current row (e.g. ranking within a department) without collapsing rows the way **groupBy** does.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank,
row_number

dept_window =
Window.partitionBy("department") \
.orderBy(col("sales_amount").desc())

ranked = df.withColumn("rank_in_dept",
rank().over(dept_window)) \
.withColumn("row_num",
row_number().over(dept_window))

ranked.select("department", "name",
"sales_amount",
"rank_in_dept", "row_num").show()
```

1. Imports the **Window** class used to define window specifications.
2. Imports **rank()** and **row_number()** window functions.
3. **Window.partitionBy("department")** groups rows per department.
4. **.orderBy(...desc())** orders rows within each partition by sales.
5. **rank().over(dept_window)** assigns a rank per department (ties share rank).
6. **row_number().over(dept_window)** assigns a unique sequential number.
7. Selects the columns needed to inspect the ranking result.
8. Displays the final ranked DataFrame.

department	name	sales_amount	rank_in_dept	row_num
Electronics	Amr Saeed	6100.00	1	1
Electronics	Youssef Ali	5200.00	2	2
Clothing	Sherif Adel	1320.75	1	1

6. Quick Reference

Method / Function	Purpose
spark.read.csv(path)	Read a CSV file into a DataFrame
df.show(n)	Display the first n rows
df.printSchema()	Print column names and data types
df.select(cols)	Choose specific columns
df.filter(cond) / df.where(cond)	Keep rows matching a condition
df.withColumn(name, expr)	Add or replace a column
df.groupBy(cols).agg(...)	Aggregate values per group
df.orderBy(col)	Sort rows by one or more columns
df.join(other, on, how)	Combine two DataFrames
df.createOrReplaceTempView(name)	Register df for SQL queries
spark.sql(query)	Run a SQL string, returns a DataFrame
Window.partitionBy(...).orderBy(...)	Define a window for ranking/analytics

End of guide — generated as a companion PDF alongside `employee_sales.csv`.